

PROJECT: Scalable Microservices Architecture with ECS and Fargate

Services: ECS, Fargate, ECR, RDS, CloudWatch

Description: Containerize a microservices-based application using Docker. Deploy using ECS Fargate and manage logs with CloudWatch. Use RDS as the backend

database.

Skills: Containers, microservices, monitoring

Definition: This project demonstrates the deployment of a containerized microservice application using Docker and AWS cloud services. A Node.js Express application is packaged into a Docker container and stored in Amazon Elastic Container Registry (ECR). The container is deployed using Amazon ECS with Fargate, allowing the application to run without managing servers. The service is exposed through a public IP and monitored using AWS CloudWatch logs.

Scope of the Project: The scope of this project focuses on building and deploying a scalable microservice architecture in the cloud using AWS services. It covers containerization of applications, storing images in ECR, and deploying containers using ECS Fargate. The project also includes monitoring application logs through CloudWatch for troubleshooting and performance monitoring. This approach can be extended to deploy multiple microservices and integrate additional services such as databases, load balancers, and CI/CD pipelines.

Methodologies:

1. **Method Used:** This project uses the Containerization and Microservices Deployment method using Docker with AWS ECS Fargate to deploy the Node.js application in a scalable cloud environment.

Why This Method is Best: This method is efficient because Docker ensures consistent application environments, while ECS Fargate automatically manages the infrastructure, making deployment scalable, reliable, and serverless.

2. **AWS EC2:** AWS EC2 requires manual server management, scaling, and maintenance, which increases operational complexity compared to serverless container services like ECS Fargate.

Services Used in this Project:

1. **Amazon EC2 (Elastic Compute Cloud)**
 - **Definition:** Amazon EC2 is a cloud service that provides virtual servers to run applications on AWS.
 - **Purpose:** EC2 was used as a development environment to build, test, and push the Docker image to Amazon ECR
 - **Role in this Project:** EC2 acted as the host machine where the Node.js application and Docker commands were executed before deploying to ECS Fargate.
2. **Fargate**
 - **Definition:** AWS Fargate is a serverless compute engine used to run Docker containers without managing servers.
 - **Purpose:** Fargate is used to deploy and run the containerized Node.js microservice automatically in AWS ECS.
 - **Role in this Project:** Fargate acts as the runtime environment that executes the Docker container of the microservice in the cloud.

3. ECR (Elastic Container Registry)

- **Definition:** Amazon ECR is a secure AWS service used to store and manage Docker container images.
- **Purpose:** ECR is used to store the Docker image of the Node.js microservice before deployment.
- **Role in this Project:** ECR acts as the container image repository from which ECS Fargate pulls the Docker image to run the application.

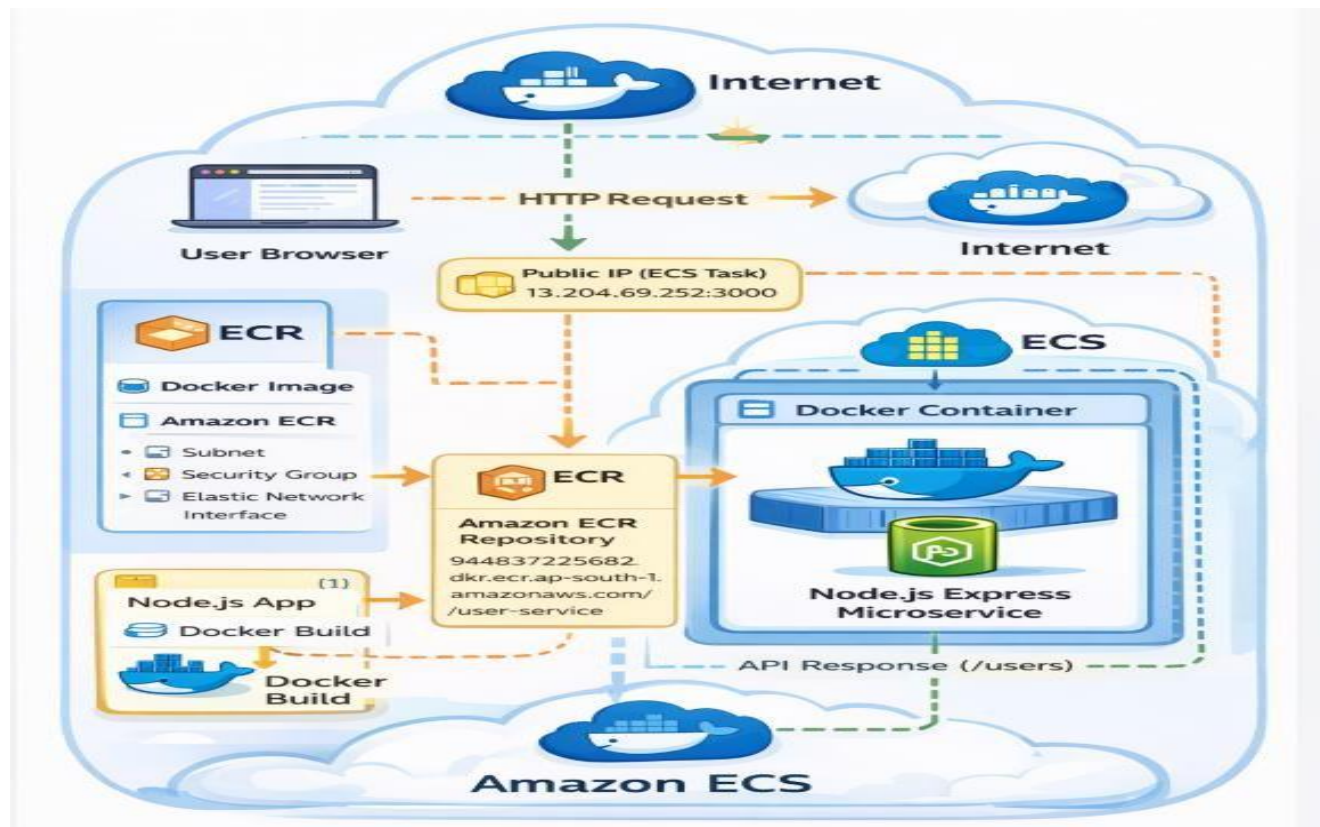
4. RDS (Relational Database Service)

- **Definition:** Amazon RDS is a managed AWS service used to run and manage relational databases in the cloud.
- **Purpose:** RDS is used to store and manage application data for the microservices.
- **Role in this Project:** RDS acts as the backend database where the microservice stores and retrieves user data.

5. CloudWatch

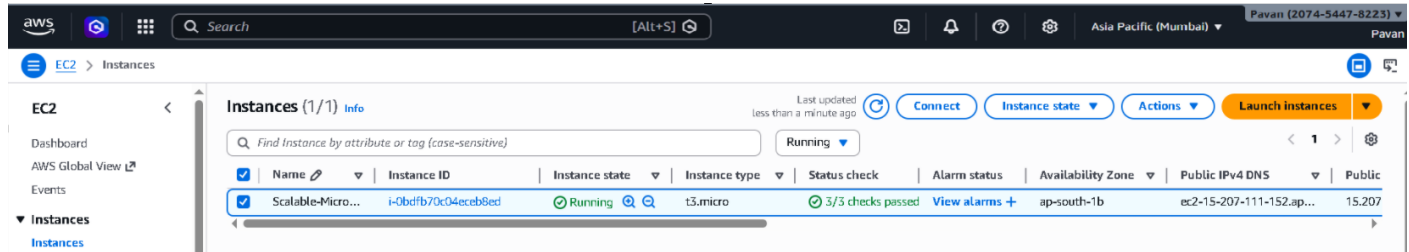
- **Definition:** Amazon CloudWatch is an AWS monitoring service used to collect and view logs, metrics, and performance data of applications and resources.
- **Purpose:** CloudWatch is used to monitor the logs and performance of the microservice running in ECS Fargate.
- **Role in this Project:** CloudWatch acts as the logging and monitoring system that captures and displays the container logs generated by the application.

Architecture Diagram:



Implementation and Execution of Scalable Microservices Architecture with ECS and Fargate.

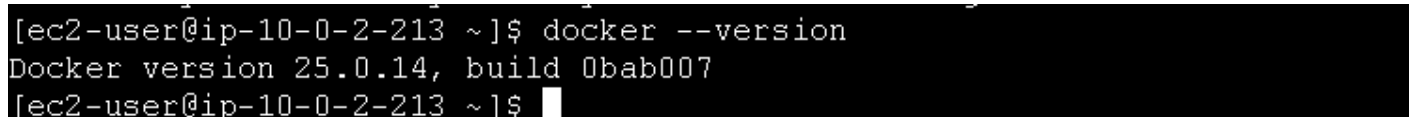
STEP 1: Launch an instance



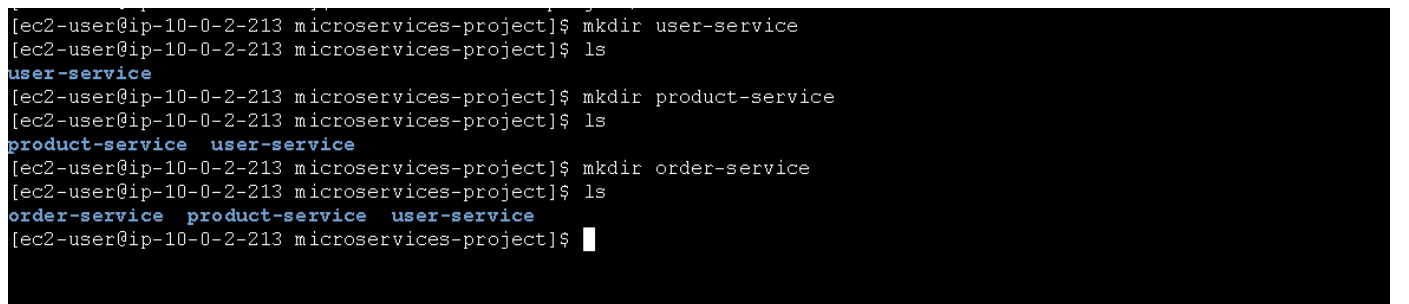
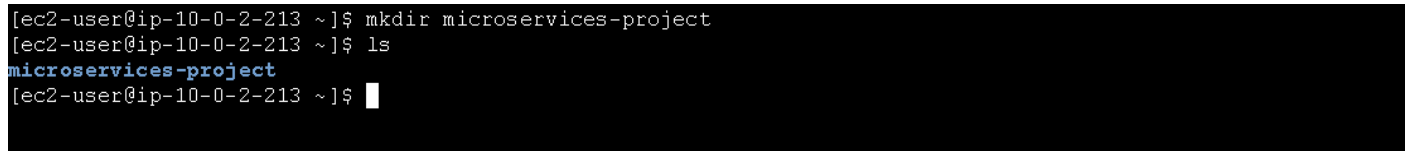
STEP 2: Update the Server and Install Docker



STEP 3: Verify Docker

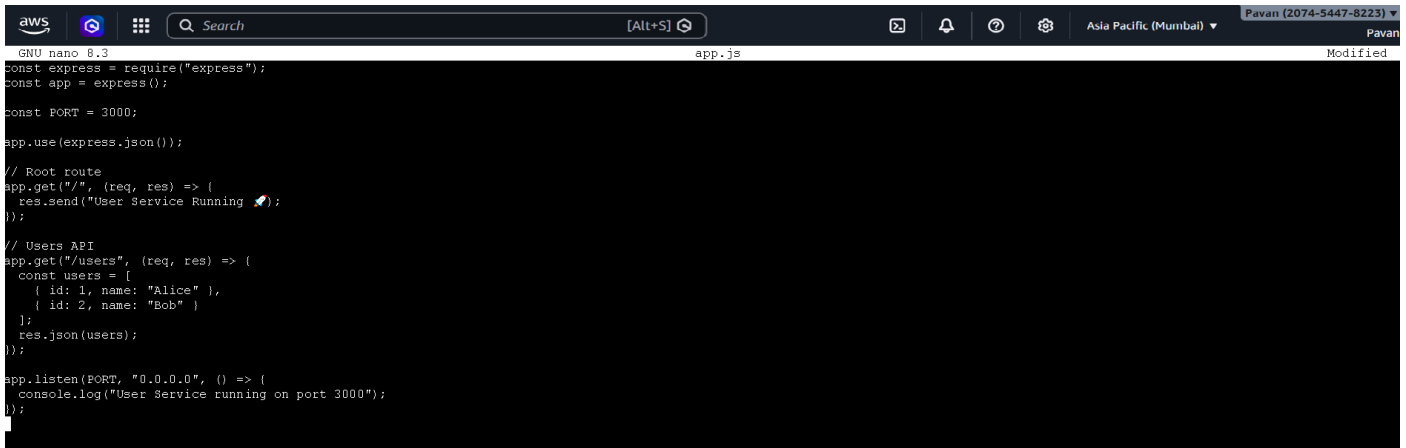


STEP 4: Create a directory name Microservices Project



STEP 5: Create application file

```
[ec2-user@ip-10-0-2-213 user-service]$ nano app.js
[ec2-user@ip-10-0-2-213 user-service]$
```



```
GNU nano 8.3 app.js Modified
const express = require("express");
const app = express();

const PORT = 3000;

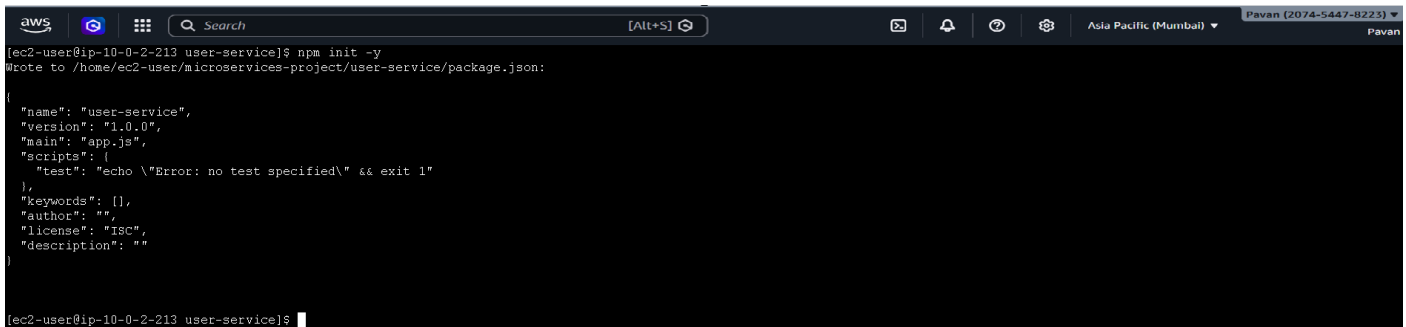
app.use(express.json());

// Root route
app.get("/", (req, res) => {
  res.send("User Service Running 🚀");
});

// Users API
app.get("/users", (req, res) => {
  const users = [
    { id: 1, name: "Alice" },
    { id: 2, name: "Bob" }
  ];
  res.json(users);
});

app.listen(PORT, "0.0.0.0", () => {
  console.log("User Service running on port 3000");
});
```

STEP 6: Install Dependencies



```
aws [Search] [Alt+S] Asia Pacific (Mumbai) Pavan (2074-5447-8223) Pavan
[ec2-user@ip-10-0-2-213 user-service]$ npm init -y
Wrote to /home/ec2-user/microservices-project/user-service/package.json:

{
  "name": "user-service",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "test": "echo \\\"Error: no test specified\\\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}

[ec2-user@ip-10-0-2-213 user-service]$
```

```
[ec2-user@ip-10-0-2-213 user-service]$ npm install express
```

```
added 65 packages, and audited 66 packages in 3s
```

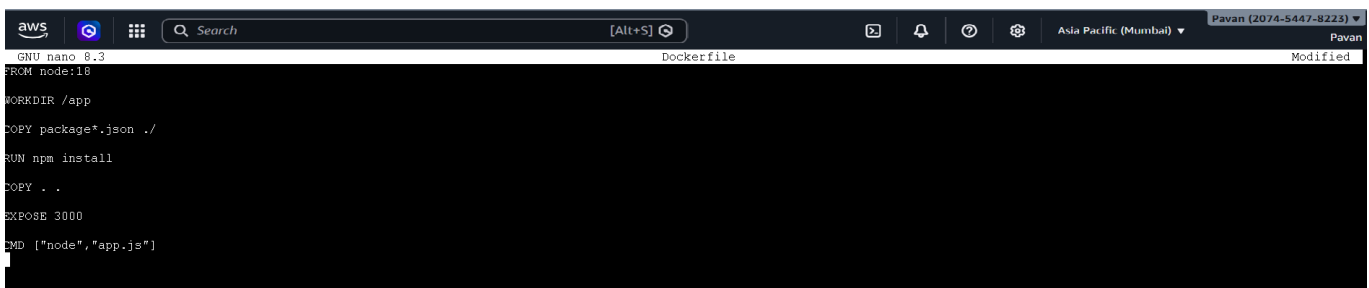
```
22 packages are looking for funding
  run `npm fund` for details
```

```
found 0 vulnerabilities
```

```
[ec2-user@ip-10-0-2-213 user-service]$
```

STEP 7: Create Dockerfile

```
[ec2-user@ip-10-0-2-213 user-service]$ nano Dockerfile
[ec2-user@ip-10-0-2-213 user-service]$
```



```
aws [Search] [Alt+S] Asia Pacific (Mumbai) Pavan (2074-5447-8223) Pavan
GNU nano 8.3 Dockerfile Modified
FROM node:18

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["node", "app.js"]
```

STEP 8: Build Docker Image

```
[ec2-user@ip-10-0-2-213 user-service]$ docker build -t user-service .
(*) Building 25.4s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 210B
=> [internal] load metadata for docker.io/library/node:18
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/node:18@sha256:c6ae79e38498325db67193d391e6ecd1224d96c693a8a4d943498556716d3783
=> => resolve docker.io/library/node:18@sha256:c6ae79e38498325db67193d391e6ecd1224d96c693a8a4d943498556716d3783
=> => sha256:c6ae79e38498325db67193d391e6ecd1224d96c693a8a4d943498556716d3783 6.41kB / 6.41kB
=> => sha256:eb2936371ee2859fad6a3c5af88d4abc6ff74399adbb13b7de3c1f11bdee6b9 2.49kB / 2.49kB
=> => sha256:b50082bc3670d0396b2d90e4b0e5bb10265ba5d0ee16bf40f9a505f7045ee563 6.39kB / 6.39kB
=> => sha256:37927ed901b1b2608b72796c6881bf645480268eca4ac9a37b9219e050bb4d84 24.02MB / 24.02MB
=> => sha256:3e6b9d1a95114e19f12262e4e8a59ad1d1a10ca7b82108adcf0605a200294964 40.49MB / 40.49MB
=> => sha256:79b2f47ad4443652b9b5cc81a95ede249fd976310efdbee159f29638783778c0 64.40MB / 64.40MB
=> => sha256:e23f099911d692f62b851cf49a1e93294288a115f5cd2d014180e4d3684d34ab 211.36MB / 211.36MB
=> => extracting sha256:3e6b9d1a95114e19f12262e4e8a59ad1d1a10ca7b82108adcf0605a200294964
=> => sha256:cda7f44f2bd4cc4bb7514474024b3f3705de00ddb6355a33be5ac7808e5b7125 3.32kB / 3.32kB
=> => sha256:c6b30c3f16966552af10ac00521f60355b1fcf446ac1c20b1038587e28583ce7 45.60MB / 45.60MB
=> => sha256:3697be50c98b9d071df4637e1d3491d00e7b9f3a732768c876d82309b3c5a145 1.25MB / 1.25MB
=> => sha256:461077a72fb7fe40d34a37d6a1958c4d16772d0dd77ff572ec50a1fdc41a3754d 446B / 446B
=> => extracting sha256:37927ed901b1b2608b72796c6881bf645480268eca4ac9a37b9219e050bb4d84
=> => extracting sha256:79b2f47ad4443652b9b5cc81a95ede249fd976310efdbee159f29638783778c0
=> => extracting sha256:e23f099911d692f62b851cf49a1e93294288a115f5cd2d014180e4d3684d34ab
=> => extracting sha256:cda7f44f2bd4cc4bb7514474024b3f3705de00ddb6355a33be5ac7808e5b7125
=> => extracting sha256:c6b30c3f16966552af10ac00521f60355b1fcf446ac1c20b1038587e28583ce7
=> => extracting sha256:3697be50c98b9d071df4637e1d3491d00e7b9f3a732768c876d82309b3c5a145
=> => extracting sha256:461077a72fb7fe40d34a37d6a1958c4d16772d0dd77ff572ec50a1fdc41a3754d
[internal] load build context
=> => transferring context: 2.21MB
=> [2/5] WORKDIR /app
=> [3/5] COPY package*.json ./
=> [4/5] RUN npm install
=> [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => writing image sha256:7da2fc5d06fadb74ee45195e3dd56elc67410856a0b8ecfe68bca9f89f4aa6f8
=> => naming to docker.io/library/user-service
[ec2-user@ip-10-0-2-213 user-service]$
```

```
[ec2-user@ip-10-0-2-213 user-service]$ docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
user-service         latest              7da2fc5d06fa       2 minutes ago      1.09GB
[ec2-user@ip-10-0-2-213 user-service]$
```

STEP 9: docker ps

```
[ec2-user@ip-10-0-2-213 user-service]$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
a13231a1d36e   user-service "docker-entrypoint.s..." 20 minutes ago Up 20 minutes 0.0.0.0:3000->3000/tcp, :::3000->3000/tcp loving_elion
[ec2-user@ip-10-0-2-213 user-service]$
```

STEP 10: Test the API

← → ↻ ⚠ Not secure 15.207.111.152:3000/users

Pretty-print ☐

{ "message": "User Service Running" }

STEP 11: Create ECR Repository

```
[ec2-user@ip-10-0-2-213 user-service]$ aws ecr create-repository \
--repository-name user-service \
--region ap-south-1
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:ap-south-1:944837225682:repository/user-service",
    "registryId": "944837225682",
    "repositoryName": "user-service",
    "repositoryUri": "944837225682.dkr.ecr.ap-south-1.amazonaws.com/user-service",
    "createdAt": "2026-03-08T06:55:16.894000+00:00",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": false
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}
[ec2-user@ip-10-0-2-213 user-service]$
```

STEP 12: Login Docker to ECR

```
[ec2-user@ip-10-0-2-213 user-service]$ aws ecr get-login-password --region ap-south-1 | \
docker login --username AWS \
--password-stdin 944837225682.dkr.ecr.ap-south-1.amazonaws.com
WARNING! Your password will be stored unencrypted in /home/ec2-user/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[ec2-user@ip-10-0-2-213 user-service]$
```

STEP 13: Tag the Docker Image

```
[ec2-user@ip-10-0-2-213 user-service]$ docker tag user-service:latest \
944837225682.dkr.ecr.ap-south-1.amazonaws.com/user-service:latest
[ec2-user@ip-10-0-2-213 user-service]$
```

STEP 14: Push the Image

```
[ec2-user@ip-10-0-2-213 user-service]$ docker push \
944837225682.dkr.ecr.ap-south-1.amazonaws.com/user-service:latest
The push refers to repository [944837225682.dkr.ecr.ap-south-1.amazonaws.com/user-service]
a64239197b93: Pushed
b323c6ee3505: Pushed
dd200bf816c1: Pushed
064f77450774: Pushed
d2a991bcab4d: Pushed
b624aa2d5ea2: Pushed
d399c9dc306f: Pushed
04f9fa179c1b: Pushed
ce84ba212e49: Pushed
e4dc8cd9ecc8: Pushed
6428cc293366: Pushed
2f7436e79a0b: Pushed
latest: digest: sha256:5de1d6c5968962c115dde0fc06eb6f63a0ca20ce9daa11ab0442710156c23d47 size: 2839
[ec2-user@ip-10-0-2-213 user-service]$
```

STEP 15: Create ECS Cluster

Amazon Elastic Container Service

Express Mode

Clusters

Namespaces

Task definitions

Account settings

Cluster microservices-cluster has been created successfully. [View cluster](#)

Clusters (1) [Info](#)

Search clusters

By default, we only load up to 1,000 clusters at a time.

Cluster	Services	Tasks	Container instances	CloudWatch monitoring	Capacity provider strategy
microservices-cluster	0	No tasks running	0 EC2	Default	No default found

STEP 16: Create Task Definition

Amazon Elastic Container Service

Express Mode

Clusters

Namespaces

Task definitions

Account settings

Amazon ECR

Repositories

AWS Batch

Task definition successfully created user-service-task1 has been successfully created. You can use this task definition to deploy a service or run a task. [View task definition](#)

user-service-task:1

Last updated March 8, 2026, 13:32 (UTC+5:30) [Deploy](#) [Actions](#) [Create new revision](#)

Overview [Info](#)

ARN arn:aws:ecs:ap-south-1:944837225682:task-definition/user-service-task:1	Status ACTIVE	Time created March 8, 2026, 13:32 (UTC+5:30)	App environment Fargate
Task role -	Task execution role ecsTaskExecutionRole	Operating system/Architecture Linux/X86_64	Network mode awsipc
Fault injection Turned off			

STEP 17: Create ECS Service

Amazon Elastic Container Service

Express Mode

Clusters

Namespaces

Task definitions

Service user-service has been successfully created. You can use this service to deploy a task definition or run a task. [Delete service](#) [Update service](#)

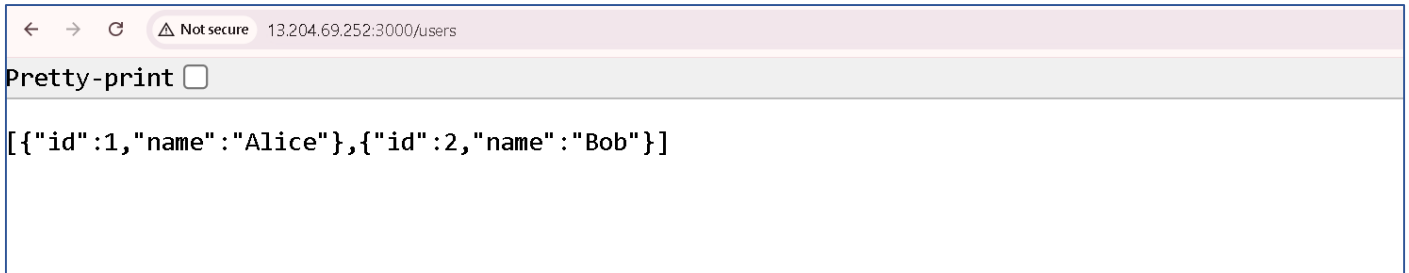
user-service [Info](#)

Last updated March 8, 2026, 16:07 (UTC+5:30)

Service overview [Info](#)

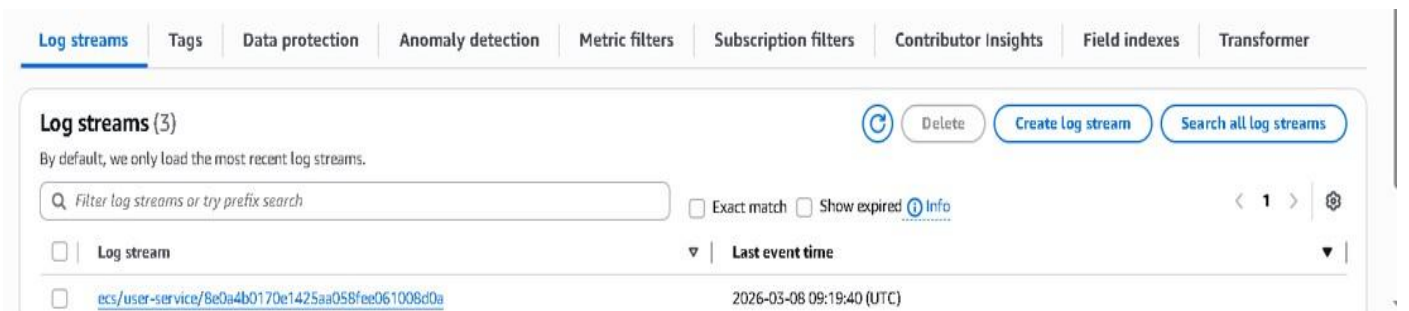
Status Active	Tasks (1 Desired) 0 Pending 1 Running	Task definition: revision user-service-task:1	Deployment status Success
-------------------------	---	---	-------------------------------------

STEP 18: Access the Service

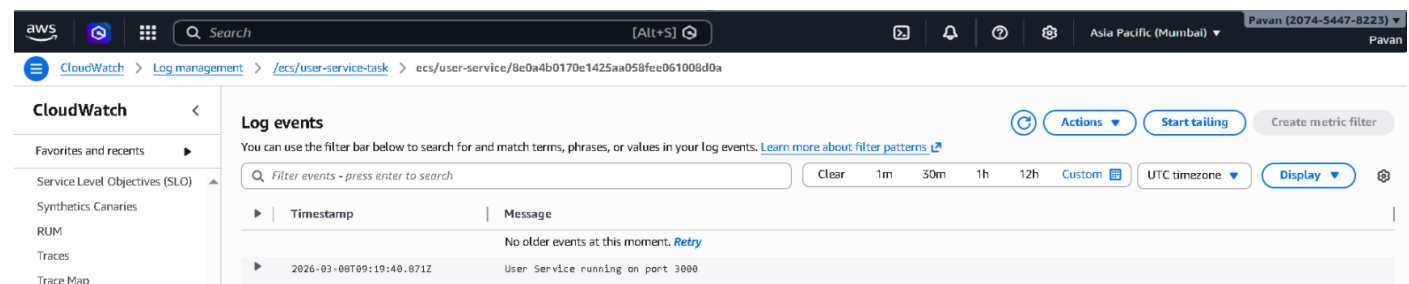


STEP 19: Monitoring Application Logs using AWS CloudWatch

➤ View Log Streams



➤ View Log Events



Troubleshooting:

1. Docker Installation and Configuration Issues
 - **Issue:** Docker was not initially configured correctly on the EC2 instance, causing errors while building the container image.
 - **Solution:** Docker was installed and configured properly on the EC2 instance using the required commands and the Docker service was started to enable container creation.

2. Authentication Error While Pushing Image to ECR

- **Issue:** Docker images could not be pushed to Amazon ECR due to authentication errors.
- **Solution:** AWS CLI authentication was configured using the `aws ecr get-login-password` command to securely log in to the ECR repository before pushing the image.

Conclusion:

In this project, a Node.js microservice was successfully containerized using Docker and deployed on AWS using ECS Fargate. The Docker image was stored in Amazon ECR and the application was made accessible through a public IP address. AWS CloudWatch was used to monitor and manage the application logs. This project demonstrates how containerization and cloud services can be used to build scalable and efficient microservice-based applications.